

PIPELINE_TRACE.md

PHASE 1 — grounded, READ-ONLY runtime trace. No product/pipeline code was changed in this phase.

Every claim below is anchored to `file:line` at repo HEAD.

- Repo: `Foltum-Studio-WEB`, branch `main`
- HEAD: `47417703afdbe31707dd47a6164c1ea54a5c625d`
- Stack: Next.js App Router / TypeScript / Prisma → Postgres (Neon)
- Generation unit = **SHOT**; strict sequential per-shot chain; camera decided only at the shot layer.

0. How to read this document

- Line numbers are at the HEAD commit above. They will shift after edits — **re-grep before quoting in later phases.**
- “Producer” = the function that WRITES a DB field/artifact. “Consumer” = the function that READS it. Coordinated producer+consumer changes are flagged explicitly in §10 (Planned edits by phase).

1. Runtime path map (idea → assembly)

Each transition lists: **entrypoint / worker fn, INPUT source, OUTPUT persistence (Prisma model.field).**

All field names cross-checked against `prisma/schema.prisma`.

1.1 idea → synopsis + drama bible

- **Entry:** `app/api/ai/idea/route.ts` (POST) — creates a `GenerationJob` of type `"synopsis"`, runs in background.
- **Worker:** `lib/workers/synopsis-job.ts` `runSynopsisJob`.
- Drama bible is generated FIRST: `generateDramaBible(...)` (`synopsis-job.ts:104`) wrapped in a **best-of-N critic loop** `generateBestOfN<GenerateDramaBibleResult>` (`synopsis-job.ts:116`), critique via `critiqueCandidate(chatJSON, "dramaBible", rendered)` (`synopsis-job.ts:125`). Imports at `synopsis-job.ts:19,22`.
- Synopsis then produced through an attempt loop with `normalizeIdeaResult` (`synopsis-job.ts:31`); language is auto-detected inside `normalizeIdeaResult` (`synopsis-job.ts:71`).
- **INPUT:** raw idea text + genre/story params from the request body.
- **OUTPUT (Prisma Project):** `Project.synopsis`, `Project.language` (`schema.prisma:75`, ISO idea language = CONTEXT only), `Project.name`, `Project.dramaBible` (Json), `Project.dramaBibleVersion`;

```
stage="synopsis" .
```

Best-effort `GenerationLog` rows written via `buildGenerationLog` (`critic.ts:190`).

1.2 synopsis → season structure → per-episode scripts

- **Entry:** `app/api/ai/season/route.ts` → drives the state machine in `lib/workers/season-script-job.ts`
(`advanceSeasonJob` / `tick` ; steps: `structure` → `map` → `cast` → per-episode).
- **Season map:** `generateSeasonMap` (`lib/season-map.ts`) — `generate` → `validate` → `retry`. Validators:
`validateCellCount` (`season-map.ts:80`), `validateCellShape` (`season-map.ts:101`),
`validateConsecutiveCliffhangers` (`season-map.ts:118`), `validateMajorCadence` (`season-map.ts:144`),
`validateEscalationRising` (`season-map.ts:188`). OUTPUT: `Season.seasonMap` (`Json SeasonMapCell[]`),
`Season.seasonMapVersion`.
- **Per-episode script:** `persistEpisodeScript` (`lib/workers/season-script-job.ts:287`).
- **INPUT:** validated `EpisodeScript` (from `validateEpisode` → `validateEpisodeScript`), plus season/episode context and the previous episode's ending (see §7).
- **OUTPUT (Prisma Episode):** `Episode.script` (rendered full script text, `schema.prisma:218`), `Episode.description` (= `outline.description` ?? logline), `logline`, `cliffhanger`, `locationName` / `locationDesc`,
`arcRole`, `status="script_ready"`, `peakSceneIndex` (`schema.prisma:237`), `promptVersion`.
- **ALSO CREATES Scene rows** here (`dialogue` / `dialogueEn` / `action` / `videoPrompt` / `durationSec` / `startState` / `endState` / ...) + `SceneCharacter` + `EpisodeCharacter`. **NOTE:** these script-derived Scene rows are later DELETED and re-created by the scene-breakdown step (§1.3) — see §2.

1.3 episode approval → scene breakdown (⚠ key transition)

- **Entry:** `app/api/ai/episodes/[id]/scenes` route → `lib/workers/scenes-job.ts` `runScenesJob`.
- **Worker:** builds the LLM message via `buildUserMessage` (`scenes-job.ts:156`), runs a background JSON call (`gpt-6-astra`), then `persistScenes` (`scenes-job.ts:251`), then `updateSeasonStateForApprovedEpisode` (`scenes-job.ts:418`) and `persistShotPlanForApprovedEpisode` (§1.4).
- **INPUT (⚠ see §2):** `buildUserMessage` uses `synopsis = episode.season?.project?.synopsis` (`scenes-job.ts:164`) and `episodeBriefBlock(episode)` (`scenes-job.ts:224`), which calls `parseEpisodeSynopsis(episode.description)` (`scenes-job.ts:148`). **It does NOT read Episode.script**.
- **OUTPUT (Prisma Scene):** `persistScenes` DELETES every existing scene of the episode (`scenes-job.ts:291`), sets `Episode.videoUrl=null` (`scenes-job.ts:292`), then re-creates DRAMATIC scenes with
`number`, `durationSec`, `action`, `dialogue`, `keyProp`, `escalationBeats` (`Json`), `location(Desc)` ;
`videoPrompt` is left `null` (camera is planned one level below).

1.4 approved scenes → shot plan

- **Worker:** `lib/workers/shot-plan-persist.ts` `persistShotPlanForApprovedEpisode` → `generateShotPlan`

(lib/shot-plan.ts:299 , generate → validate → targeted retry, SHOT_PLAN_MAX_ATTEMPTS = 3 at shot-plan.ts:287).

- **INPUT:** Scene.action / dialogue / keyProp / escalationBeats + Season.seasonMap cliffhanger-Type.
- **OUTPUT (Prisma Shot):** rows with index (per scene), shotType , size , duration (Float, planned), speakerId , line , reactionOfId , escalationBeat , postFx , matchCutIn , matchCutOut , promptVersion , status="pending" .
- **Floor:** MIN_SHOTS = 15 (shot-plan-persist.ts:33); fewer than that → LOUD failure: Episode.status="shot_plan_failed" + Episode.chainRunNote (shot-plan-persist.ts:49,174-175).

1.5 shot plan → per-shot video generation (sequential chain)

- **Entry:** app/api/ai/episodes/[id]/generate-all/route.ts → chain driver lib/chain-run.ts (nextSequentialShot).
- **Worker:** lib/workers/video-job.ts runVideoJob / runShotVideoJob — submits to WaveSpeed (Seedance 2.5, generate_audio: true).
- **INPUT:** a PlannedShot reconstructed from the Shot row (video-job.ts:314-328), character portraits + location refs, prompt from assembleShotPrompt (video-job.ts:333).
- **OUTPUT (Prisma Shot):** Shot.videoUrl (schema.prisma:452), status , attempt bookkeeping.
- **Chain continuation:** continueShotChain (video-job.ts:461+) advances to the next shot; when nextSequentialShot returns null it triggers assembly (§1.6).
- ⚠ **Timing note (see §8):** the SUBMITTED duration is Math.max(4, ...) (video-job.ts:360) — the provider floor.

1.6 all shots ready → assembly

- **Worker:** lib/workers/assembly-job.ts runAssemblyJob (triggered by continueShotChain when no next shot).
- **INPUT:** every Shot of the episode joined to its Scene , flattened to an episode-global ordered list (assembly-job.ts:71-86); readiness check buildConcatPlan (assembly-job.ts:91).
- **STEPS:** assembleEpisodeLocally (ffmpeg.ts:944 , seamless hard-cut join, quiet music by closing beat) → buildSubtitleSpec → subtitleSpecToAss → burnSubtitlesFile (⚠ subtitles, §3B/§4) → upload to S3.
- **OUTPUT (Prisma Episode):** Episode.videoUrl (schema.prisma:221), status="assembled" , assembleQuality , assembleFps , chainRunActive=false (assembly-job.ts:149-158).

2. Where the APPROVED Episode.script is actually used (and where it is NOT)

FINDING — CONFIRMED: scene breakdown does NOT consume Episode.script . It re-composes from

Project.synopsis + Episode.description (outline-level), so the approved full script's scene

**content is
discarded for the video pipeline.**

Evidence (lib/workers/scenes-job.ts buildUserMessage , verified at HEAD):

- scenes-job.ts:164 — const synopsis = episode.season?.project?.synopsis ?? "";
- scenes-job.ts:224 — user message embeds \${episodeBriefBlock(episode)} .
- scenes-job.ts:148 — episodeBriefBlock → const { synopsis, cliffhanger } = parseEpisodeSynopsis(episode.description);
- scenes-job.ts:150 — the brief is built from episode.description (the outline synopsis + cliffhanger), never episode.script .
- The Prisma include at scenes-job.ts:157-160 loads season.project only; Episode.script is not even selected here.

Where Episode.script IS read downstream:

- lib/workers/scenes-job.ts:427 — updateSeasonStateForApprovedEpisode folds episode.script into SeasonState (season memory; §7). This is the one pipeline consumer of the approved script.
- lib/workers/artifact-images-job.ts:68-71 — extracts artifacts from the script.
- lib/storyboard-dialogue.ts:268,298 — storyboard fallback script || description .
- UI display only: season-stage.tsx , episode-view.tsx:1043 (BookScript), and the revise route.

Consequence: persistEpisodeScript (§1.2) creates Scene rows FROM the script, but scene-break-down

persistScenes DELETES them (scenes-job.ts:291) and re-creates scenes from Episode.description . The narrative

detail a user reviewed/approved in Episode.script never reaches the shot planner or the video model.

3. Confirm/deny at HEAD (quoted code)

3A — scenes-job builds scenes from description without Episode.script ; persistScenes slices extra scenes & only logs incomplete; updateSeasonStateForApprovedEpisode saves regardless of validity

ALL CONFIRMED.

- Builds from description not script: see §2 (scenes-job.ts:148,150,164,224).
- Slices extras: scenes-job.ts:259 — const trimmed = rawScenes.slice(0, MAX_SCENES); (MAX_SCENES = EPISODE_MAX_SCENES at scenes-job.ts:84).
- Only LOGS incompleteness (no gate/throw): scenes-job.ts:279 computes missingDrama , and scenes-job.ts:280-285 merely console.log(...) . There is no validity gate before persisting scenes.
- Saves regardless of validity: scenes-job.ts:458-459 comment “Persist regardless of valid flag...”, then prisma.seasonState.create(...) at scenes-job.ts:460-467 .

3B — assembly-job generates + burns subtitles; assembleEpisodeLocally receives clip URLs WITHOUT planned durations

BOTH CONFIRMED.

- Burns subtitles: assembly-job.ts:127-142 — buildSubtitleSpec(ordered, { dialogueLanguage })

(`assembly-job.ts:129`) → `subtitleSpecToAss` written to `subs.ass` (`assembly-job.ts:132-133`) → `burnSubtitlesFile(assembled.outputPath, assPath, subbedPath)` (`assembly-job.ts:135`).

- No planned durations passed to the joiner: `assembly-job.ts:107-108` calls

`assembleEpisodeLocally(ordered.map((s) => ({ videoUrl: s.videoUrl as string, audioUrl: s.audioUrl })), ...)`.

Only `videoUrl` / `audioUrl` are passed; the `duration` field carried in `ordered` (`assembly-job.ts:77`) is used

ONLY for subtitle cue timing in `buildSubtitleSpec`, never for trimming.

- `assembleEpisodeLocally` signature confirms: `SceneClipInput[]` (`ffmpeg.ts:944`); it concatenates FULL clips

(seamless hard cut, `ffmpeg.ts:1006-1017`) and does NOT trim to any planned per-shot duration.

4. Complete subtitle inventory (every path)

POLICY (record for later phases): subtitles must be COMPLETELY REMOVED from the product + generation.

The earlier “centered subtitle burn” decision is **REVERSED**. Deletion is Phase 2; this section is the full map.

△ `language` / `dialogueLanguage` type fields may drive SPEECH, NOT subtitles — see §4.3; do NOT delete those blindly.

4.1 Subtitle text generation, ASS/SRT/VTX, ffmpeg burn

- `lib/shot-pipeline.ts` — subtitle machinery:
- `SubtitleCue` interface (`shot-pipeline.ts:99`), `SubtitleSpec` interface (`shot-pipeline.ts:108-109`).
- `buildSubtitleSpec(shots, opts)` (`shot-pipeline.ts:123`) — CENTERED cues from spoken lines; language default
“en” (`shot-pipeline.ts:127`); cue timing accumulates each shot’s clip duration (`shot-pipeline.ts:119`).
- `subtitleSpecToAss(spec, opts)` (`shot-pipeline.ts:162`) — ASS document; Format line at `shot-pipeline.ts:174` ; **Alignment=2 = bottom-center** (`shot-pipeline.ts:175` , comment at `shot-pipeline.ts:157`).
- Doc header describing it: `shot-pipeline.ts:10` .
- `lib/ffmpeg.ts` — the burn:
- `burnSubtitlesFile(inputPath, assPath, outputPath)` (`ffmpeg.ts:1112`) — `ffmpeg -vf subtitles=${base} filter` (`ffmpeg.ts:1116`).
- (Unrelated) `ffmpeg.ts:1032` comment “No subtitles are burned in” refers to `assembleEpisodeLocally`’s OWN join step; the actual burn happens afterward in `assembly-job.ts` (§3B).
- `lib/workers/assembly-job.ts` — orchestration: doc `assembly-job.ts:12-14` ; imports `burnSubtitlesFile` (`assembly-job.ts:25`), `buildSubtitleSpec` / `subtitleSpecToAss` (`assembly-job.ts:35-36`); `dialogueLanguage` read (`assembly-job.ts:100`); burn block (`assembly-job.ts:127-142`); log (`assembly-job.ts:162`).

4.2 Model-prompt instructions (both burn-related directives AND negatives)

- `lib/prompts/shot.ts` — `dialogueLanguage` import (`prompts/shot.ts:24`), doc (`prompts/shot.ts:102-106`),
burned-subtitle language handling (`prompts/shot.ts:176-183`), negative “no on-screen text or captions”
(`prompts/shot.ts:214`).
- `lib/prompts/scene.ts` — `dialogueLanguage` + “no on-screen text or captions” (`prompts/scene.ts:15,124,284,393`).
- `lib/season.ts` — comments “UI/subtitles” (`season.ts:453,1153,1197`); `SceneScriptInput.dialogueLanguage`
(`season.ts:1453-1457`); `dialogueLanguageDirective` (`season.ts:1501`).
- `lib/workers/season-script-job.ts` — comments “UI + subtitles” (`season-script-job.ts:306,313`),
`subtitled:false` (`season-script-job.ts:348`), `dialogueLanguage` (`season-script-job.ts:671`).
- `lib/workers/video-job.ts` — import `getDialogueLanguage` (`video-job.ts:24`), passed to prompt
(`video-job.ts:338`), doc “burned subtitles” (`video-job.ts:449`).
- `lib/scene-prompt.ts:493` — negative “no subtitles or captions”.
- `lib/voiceover.ts` — “No subtitles” negatives (`voiceover.ts:7,43,158,162,202,210,217`).
- Other prompt negatives (“no captions / no on-screen text”): `lib/storyboard-prompt.ts:180` ,
`lib/region-plate.ts:109` , `lib/reangle.ts:81` , `lib/keyframe.ts:151` , `lib/prompts/cast.ts:181-182` .

4.3 language / dialogueLanguage fields — SPEECH vs SUBTITLE classification

Field (file:line)	Role	Evidence
Project.language (schema.prisma:75)	CONTEXT (idea/synopsis language, ISO)	schema comment; used as “context only” in prompts (scenes-job.ts:206-211)
Project.dialogueLanguage (schema.prisma:80 , default "en")	SPEECH (dialogue/names/titles generated in this language; also currently drives burned subtitles + stored lines)	schema comment schema.prisma:78-80
Scene.language (schema.prisma:377 , default "en")	SPEECH (“spoken language for native audio”)	schema comment
Scene.dialogue vs Scene.dialogueEn (schema.prisma:368-369)	dialogueEn = SPEECH (English lines spoken by Seedance); dialogue = story-language text for UI/subtitle display	schema comment schema.prisma:369
Scene.subtitled (schema.prisma:370 , Boolean default false)	SUBTITLE state flag (“subtitles already burned into videoUrl”)	schema comment
Shot.line (schema.prisma:442)	SPEECH (the spoken English line)	schema comment
Shot.lineTranslation (schema.prisma:443)	SUBTITLE (“translation for burned subtitles; English == line for now”)	schema comment
lib/dialogue-language.ts	resolver/labels for SPEECH language: normalizeDialogueLanguage (:48), getDialogueLanguage (:62), isEnglish (:67), dialogueLanguageLabel (:72), dialogueLanguageDirective (:82)	—

Rule for Phase 2: delete the SUBTITLE role (burn machinery, Scene.subtitled , Shot.lineTranslation , the centered-caption prompt directives, the <track> /preview UI) but PRESERVE the SPEECH role

(`dialogueLanguage` , `Scene.language` , `dialogueEn / line`) unless the product also drops multi-language speech.

4.4 UI toggles / preview / export / player <track>

- `app/project/[id]/_components/scenes-stage.tsx:173` — comment (no-subtitles).
- `app/project/[id]/_components/season-stage.tsx:129` — comment.
- `app/api/ai/scenes/[id]/prompt/block/route.ts:153-154` — `dialogueLanguage: null` .
- `app/api/ai/scenes/[id]/revise/route.ts:59,66,90` — `subtitled` flag handling.
- (No HTML <track> element or WebVTT export path was found — subtitles are burned-in only, not a sidecar track.)

5. QC machinery (critics / validators / retries)

5.1 Synopsis / drama-bible path (`lib/workers/synopsis-job.ts` , `lib/critic.ts`)

- Drama bible: **best-of-N** with critique. `generateBestOfN<T>` (`critic.ts:276`), `critiqueCandidate` (`critic.ts:238`), `pickBestVariant` (`critic.ts:129`), `aggregateScore` (`critic.ts:78`), `actionForOverall` (`critic.ts:88`). Invoked at `synopsis-job.ts:116,125` .
- Synopsis: attempt loop + `normalizeIdeaResult` (`synopsis-job.ts:31`).
- **Gates:** best variant is selected; failures throw only if EVERY variant failed (`critic.ts:291` “generateBestOfN: every variant failed to generate”). Otherwise best-effort output proceeds.

5.2 Season structure / map / episode-script path (`lib/workers/season-script-job.ts`)

- `MAX_ATTEMPTS = 2` (`season-script-job.ts:129`); `retryDecision` (`season-script-job.ts:188-189`).
- `validateStructure` (`season-script-job.ts:216`) → `validateEpisodeSynopses` (`season-script-job.ts:225`).
- `validateFullStory` (`season-script-job.ts:231`).
- `validateEpisode` (`season-script-job.ts:238`) → `validateEpisodeScript` (`season-script-job.ts:254` , from `lib/season.ts`), with `episodeRetryNote` (`season-script-job.ts:263`) and English enforcement `forceEnglishDialogue` (`season-script-job.ts:269`).
- Season map validators: see §1.2 (`season-map.ts:80,101,118,144,188`), `generate` → `validate` → `retry`.
- **Gates:** retry up to `MAX_ATTEMPTS`; the final attempt softens language checks (`season-script-job.ts:254` `languageIsSoft: !!opts.finalAttempt`) and still persists.

5.3 Scene-breakdown path (`lib/workers/scenes-job.ts`)

- ⚠ **NO drama-completeness gate.** `missingDrama` is computed and only `console.log` ‘d (`scenes-job.ts:279-285`); scenes persist regardless (§3A).

5.4 Shot-plan path (`lib/shot-plan.ts`)

- `validateShotPlan` (`shot-plan.ts:203`) aggregates ALL rule validators (`shot-plan.ts:205-212`).

- `generateShotPlan` (`shot-plan.ts:299`) — `generate` → `validate` → `targeted retry`,
`SHOT_PLAN_MAX_ATTEMPTS = 3`
(`shot-plan.ts:287` , loop `shot-plan.ts:310-318`). Returns best-effort last list even if still invalid
(`shot-plan.ts:319`).
- **Gate:** `persistShotPlanForApprovedEpisode` enforces `persisted < MIN_SHOTS` →
`shot_plan_failed`
(`shot-plan-persist.ts:173-175`). This is the ONLY hard downstream gate that blocks video generation.

6. Mandates (dramaturgical rules) — file:line

- **Escalation ladder (fixed order):** `ESCALATION_LADDER` (`lib/season.ts:111`), type `Escalation-Step`
(`season.ts:119`). Contents (verbal → physicalLight → symbolic → physicalHeavy → statusReveal → thirdForce).
Beats per scene: `SCENE_ESCALATION_MIN_BEATS = 5` (`season.ts:100`),
`SCENE_ESCALATION_MAX_BEATS = 7`
(`season.ts:101`). Scene prompt mandates 5–7 consecutive PREFIXED rungs with the “symbolic” rung acting on the
`keyProp` (`scenes-job.ts:226`). Shot-layer beat validator: `validateEscalationRising` (`season-map.ts:188`) at the
season-map level; shot escalation checked inside `validateShotPlan` .
- **Exactly one prop per scene:** scene prompt “keyProp = one meaningful physical object” (`scenes-job.ts:226`);
shot-plan prompt “KEY PROP” (`lib/prompts/shot-plan.ts:167`).
- **Two-way dialogue per scene:** scene prompt “a real two-way ENGLISH dialogue exchange ... the characters
answering each other” (`scenes-job.ts:226`); episode-script rule R3 (`season.ts:1349`).
- **Fixed frame / shot scheme:** `lib/prompts/shot-plan.ts:101-135` (SHOT COUNT 15–30, OPENING first-3-no-
establishing, REACTIONS after HIGH lines, SILENCE ≥30%, LINES ≤12 words, VARIETY no adjacent size+camera,
ESCALATION, DIALOGUE FRAMING no WS while speaking); `SHOT_TYPES` (`prompts/shot-plan.ts:21`); sizes CU/MCU/MS/WS.
Enforced by the validators in `lib/shot-plan.ts:75-212` .
- **Scene / shot COUNT as a quality gate:**
- Shots: `validateShotCounts` rejects `< EPISODE_MIN_SHOTS(15)` or `> EPISODE_MAX_SHOTS(30)`
(`shot-plan.ts:180-183` ; constants `season.ts:94-95`). Persist floor `MIN_SHOTS = 15`
(`shot-plan-persist.ts:33,174`) → `shot_plan_failed` .
- Scenes: `EPISODE_MIN_SCENES = 5` / `EPISODE_MAX_SCENES = 8` (`season.ts:50-51`); script validator flags
`< MIN` as hard, `> MAX` as soft (`season.ts:745-746`); zod caps parse at
`EPISODE_MAX_SCENES + 6` headroom (`season.ts:516`). Scene-breakdown slices extras to MAX (§3A).

7. Season memory (SeasonState)

- **Model:** `SeasonState (schema.prisma:192-197): seasonId , reflectsEpisodeNumber Int? (schema.prisma:195 , “the episode number whose approval produced this state; null = seeded”), state Json (SeasonStateData: characters / props / openThreads / plantedSetups / revealedToAudience / lastSceneEndState), version String .`
 - **WRITE (producer):** `updateSeasonStateForApprovedEpisode ( scenes-job.ts:418 )` folds `episode.script ( scenes-job.ts:427 )` into the state via `generateSeasonStateUpdate ( scenes-job.ts:446 )` and does an **append-only** `prisma.seasonState.create ( scenes-job.ts:460-467 )` with `reflectsEpisodeNumber: episode.number` . Comment “Persist regardless of valid flag ... Append-only (newest row wins)” (`scenes-job.ts:458-459`).
 - **READ (current-state for folding):** `prisma.seasonState.findFirst({ where:{ seasonId }, orderBy:{ updatedAt: "desc" } }) ( scenes-job.ts:431 )` . ⚠ **The most-recent row is used regardless of reflectsEpisodeNumber** — ordering is by `updatedAt desc` , NOT by matching the predecessor episode number. So re-approving an earlier episode after a later one folds the LATER state as its “previous” context.
 - **Separate predecessor read for scripts:** `loadPreviousEnding ( season-script-job.ts:422` , per prior read of L287-460) reads the immediately-preceding episode’s last-scene `endState` + a short scene tail when writing the next episode’s script — this is a different mechanism from `SeasonState` and IS episode-adjacent.
-

8. Assembly / timing — what exists vs. what is missing

Four distinct durations, per the task’s framing:

Concept	Exists?	Where (file:line)
Planned duration	✓ in DB	Shot.duration Float? (schema.prisma:440 , “planned clip length 1.5–4; a reaction shot is 0.8–1.5”). Scene-level Scene.durationSec Int? (schema.prisma:380).
Provider-request duration	✓ derived at submit (not persisted)	video-job.ts:360 — duration: Math.max(4, Math.round(Number(params.duration ?? shot.duration ?? 3))) . Upstream, the chain computes duration = sceneClipSeconds(tier.id, shotDuration) (video-job.ts:477-478), and sceneClipSeconds clamps to [SCENE_MIN_SECONDS(3), SCENE_MAX_SECONDS] with a baseDuration floor (season.ts:1653-1657).
Actual returned file duration	✗ MISSING	No field on Shot stores the real clip length. assembleEpisodeLocally probes each clip (ffmpeg.ts:995,1035) for join math, but that value is never persisted back to the DB.
Chosen edit segment	= whole clip (no per-shot trim)	assembleEpisodeLocally concatenates FULL clips via seamless hard cut (ffmpeg.ts:1006-1017); the only trim is a sub-frame tail for frame alignment (ffmpeg.ts:1012 tailTrim), and the final -t bound (ffmpeg.ts:931) applies to the WHOLE-episode music mux, not per shot. No trim-back to Shot.duration .

△ **CORRECTION vs. the prior working note (report honestly):** the earlier assumption was that `video-job.ts` submits `Math.max(1, Math.round(shot.duration))`, so a 3 s shot would be rejected by WaveSpeed's

4 s minimum (HTTP 400). **At HEAD this is NOT the case** — the submit boundary is `Math.max(4, ...)` (`video-job.ts:360`), so the request never goes below 4 s and there is no such 400. The real timing problem is a

planned-vs-generated divergence:

1. A shot planned at 0.8–4 s (`schema.prisma:440`) is inflated by the chain (`sceneClipSeconds floor ≥3 + baseDuration`) and then floored again to **≥4 s at submit** (`video-job.ts:360`). Every short/reaction shot is therefore generated as a ≥4 s clip.
2. The **actual returned duration is never stored**, so nothing reconciles plan vs. reality.
3. Subtitle cue timing uses the PLANNED `Shot.duration` (`shot-pipeline.ts:119`, via `assembly-job.ts:77,129`), which matches neither the requested nor the real clip length → cues drift.
4. Assembly uses whole clips (no trim), so the episode's real runtime = $\Sigma(\text{provider-returned durations, each } \geq 4 \text{ s})$, which can exceed the planned `EPISODE_MIN/MAX_TOTAL_SECONDS` band (`season.ts`).

(Per task scope: no schema change is proposed here — this section only documents which durations exist and which are missing.)

9. Baseline (recorded this phase)

- `npm tsc --noEmit` → **24 errors total**. △ **Correction:** they are NOT all in `scripts/test-stage*.ts`.
Breakdown: **4 in app/**, **20 in scripts/**. The 4 app errors are pre-existing Prisma-null typing issues, unrelated to the pipeline changes planned here:
 - `app/api/ai/characters/[id]/undo/route.ts(36,24)` — TS2322 null → Json input.
 - `app/api/ai/locations/[id]/undo/route.ts(36,24)` — TS2322 null → Json input.
 - `app/api/ai/scenes/[id]/undo/route.ts(35,24)` — TS2322 null → Json input.
 - `app/api/ai/generate-episode-videos/route.ts(89,19)` — TS2322 `string | null` → `string`.
 The remaining 20 are in `scripts/` (mostly `scripts/_exp/*` and `scripts/test-stage*.ts` referencing removed/renamed exports and `.ts` import-extension issues) — treat as pre-existing noise, but do NOT claim “all errors are in test-stage files.”
- `npm next build --webpack` → exit code recorded in §9.1 below (build launched detached this phase).
- **Test runner:** there is **no unit-test framework** (no jest/vitest/mocha in `package.json`; only `tsx` is present). `package.json` scripts: `dev`, `build` (`prisma generate + prisma db execute ... patch.sql + next build --webpack`), `start`, `lint` (`eslint .`), `postinstall`. “Tests” are **110 standalone**

`scripts/test-stage*.ts` manual harness scripts (run individually with `tsx`), plus assorted `scripts/test-*.ts` (e.g. `test-assembly.ts`, `test-generation.ts`). No aggregate test command exists.

- **No paid generation was run.**

9.1 next build result

- `npm run next build --webpack` → **exit code 0** (build succeeded this phase). All routes compiled; the build ran `prisma generate + next build --webpack` cleanly (the `prisma db execute ... patch.sql` step is guarded by `|| echo 'patch.sql skipped'`). No pipeline-blocking build errors.

10. Planned edits by phase (map → specific file:line)

Producer+consumer pairs that MUST change together are flagged [COORDINATE].

Phase 2 — REMOVE subtitles completely from product + generation

- **Burn machinery (delete):**
 - `lib/workers/assembly-job.ts:127-142` (burn block), imports `assembly-job.ts:25,35-36`, `dialogueLanguage`
`read assembly-job.ts:100`, log field `assembly-job.ts:162`.
 - `lib/ffmpeg.ts:1112-1117` `burnSubtitlesFile`.
 - `lib/shot-pipeline.ts:99-195` — `SubtitleCue`, `SubtitleSpec`, `buildSubtitleSpec`, `subtitleSpecToAss`
(keep `buildConcatPlan` at `:78` and `musicForBeat` at `:194`).
- **DB fields (schema — Phase 2b, needs migration; do NOT change in Phase 2a code-only):**
 - `Scene.subtitled` (`schema.prisma:370`) — subtitle-only, safe to drop. [COORDINATE] producer `season-script-job.ts:348`, consumers `scenes/[id]/revise/route.ts:59,66,90`.
 - `Shot.lineTranslation` (`schema.prisma:443`) — subtitle-only. [COORDINATE] consumer `buildSubtitleSpec`.
- **Prompt directives (delete the caption/burn language, KEEP speech language):**
 - `lib/prompts/shot.ts:176-183` (burned-subtitle language block) — remove; keep the SPEECH `dialogueLanguage` use at `prompts/shot.ts:24,102-106,214`.
- Doc/comment cleanup: `assembly-job.ts:12-14`, `video-job.ts:449`, `season-script-job.ts:306,313`, `season.ts:453,1153,1197`, `ffmpeg.ts:1032`.
- The “no on-screen text / no captions” NEGATIVES (`prompts/shot.ts:214`, `prompts/scene.ts` various, `scene-prompt.ts:493`, `voiceover.ts:*`, `storyboard-prompt.ts:180`, `region-plate.ts:109`, `reangle.ts:81`, `keyframe.ts:151`, `cast.ts:181-182`) SHOULD STAY — they prevent the model from rendering text; that is now the ONLY subtitle policy.
- ⚠ **DO NOT touch SPEECH fields:** `Project.dialogueLanguage` (`schema.prisma:80`), `Scene.language`

(`schema.prisma:377`), `Scene.dialogueEn` (`schema.prisma:369`), `Shot.line` (`schema.prisma:442`), `lib/dialogue-language.ts` — these drive native spoken audio, not subtitles.

Phase 3 — make scene-breakdown consume the approved `Episode.script`

- `lib/workers/scenes-job.ts:156-229` `buildUserMessage` — feed `episode.script` (add to the `include / select` at `:157-160`, currently only `season.project`) instead of `/` in addition to `episode.description`.
[COORDINATE] with the drama-completeness gate (below) so the richer input is actually enforced.

Phase 4 — add a drama-completeness gate to scene-breakdown

- `lib/workers/scenes-job.ts:279-285` — convert the `missingDrama` LOG into a `validate` → `retry` → `fail` gate
(mirror `generateShotPlan` 's loop at `shot-plan.ts:310-318`).

Phase 5 — timing reconciliation

- Persist the ACTUAL returned clip duration on `Shot` (new field — schema change, deferred) and reconcile plan vs. request vs. actual. Touch points: `video-job.ts:360` (request), `assembly-job.ts:77,129` + `shot-pipeline.ts:119` (cue timing uses planned), `ffmpeg.ts:995,1035` (actual is probed but discarded).
[COORDINATE] producer (video-job persists actual) + consumer (assembly/subtitle timing reads actual).
- Decide policy for sub-4 s planned shots vs. the WaveSpeed 4 s floor (`video-job.ts:360`, `season.ts:1653-1657`).

End of PIPELINE_TRACE.md (Phase 1). No product code was modified in this phase.